

TUGAS MANDIRI 2

Data Quality Audit & Preprocessing Pipeline

“From Dirty Data to Model-Ready Data: Building a Preprocessing Pipeline”

Nama: Muhammad Rafa Aidil Gumilar

NIM: 20240801201

Mata Kuliah: Data Mining

GitHub: https://github.com/mrafaaidilgumilar26/P02_20240801201_Muhammad-Rafa-Aidil-Gumilar_Processing_Pipeline.git

A. DATA QUALITY PROBLEMS IDENTIFIED

Selama fase Data Understanding & Audit pada dataset Telco Customer Churn (7.043 baris, 21 kolom; unit analisis = satu pelanggan; target = Churn), kami mengidentifikasi beberapa masalah kualitas data. Berikut masalah-masalah utama yang ditemukan dan strategi penanganannya:

1) Missing Values

Temuan:

- TotalCharges: 11 records (0,16%) - kolom numerik yang tersimpan sebagai teks berisi spasi, sehingga isna() awalnya melaporkan 0 missing
- Semua baris tersebut memiliki tenure = 0 (pelanggan baru yang belum pernah ditagih), jadi kekosongannya bukan acak. Kolom lain: 0 missing

Dampak Bisnis: model tidak dapat dilatih pada kolom bertipe teks; mengisi dengan mean/median memalsukan tagihan pelanggan baru; deletion mengurangi sample size.

Solusi yang Diterapkan:

- TotalCharges diisi 0 jika tenure = 0 (aturan bisnis per baris, tanpa statistik lintas baris sehingga tidak menyebabkan leakage)
- Pipeline tetap memuat median imputer (numerik) dan modus (kategorikal) untuk menangani data baru yang kosong

2) Duplicate Records

Temuan: baris identik = 0; customerID berulang = 0; baris mirip jika customerID diabaikan = 0.

Dampak Bisnis: duplikat yang sebenarnya dapat membuat model overfit dan membuang resource komputasi.

Solusi yang Diterapkan: tidak ada record yang dihapus karena tidak ada duplikat exact. Hasil: dataset tetap 7.043 records.

3) Class Imbalance

Temuan: target variable (Churn) memiliki distribusi yang tidak seimbang:

- No-Churn: 5.177 records (73,51%)
- Yes-Churn: 1.866 records (26,49%) | Imbalance Ratio: 2,77:1

Dampak Bisnis: model cenderung bias ke kelas mayoritas (baseline 'selalu No-Churn' sudah 73,5% accuracy), recall churn rendah, dan false negative (churn tidak terdeteksi) sangat costly.

Solusi yang Diterapkan: class_weight='balanced' dan SMOTE (hanya pada data training).

Hasil: training data balanced dari 5.634 menjadi 8.282 records (50-50 distribution); test set dibiarkan asli.

4) Feature Scaling Requirement

Temuan: numerical features memiliki range yang sangat berbeda:

- tenure: 0 - 72 | MonthlyCharges: 18 - 118 | TotalCharges: 0 - 7.938 (skewed ke kanan)

Dampak: KNN dan algoritma berbasis distance sangat sensitif terhadap scale; TotalCharges (ribuan) akan mendominasi tenure (puluhan) dalam perhitungan jarak.

Solusi yang Diterapkan: StandardScaler (z-score) pada 3 fitur numerik, di-fit hanya pada train. Hasil: mean = 0 dan std = 1 pada data train.

5) Categorical Features

Temuan: 16 atribut kategorikal/biner: 1 ordinal (Contract), 6 biner, 9 nominal. Solusi: Contract -> OrdinalEncoder (urutan Month-to-month < One year < Two year); biner -> OneHot drop='if_binary'; nominal -> OneHot handle_unknown='ignore'; customerID dibuang.

B. PREPROCESSING PIPELINE ARCHITECTURE

Preprocessing pipeline dirancang dengan urutan yang tepat untuk menghindari data leakage dan memastikan reproducibility (ColumnTransformer + Pipeline). Tahapan yang dilakukan:

Step	Aktivitas	Alasan & Note
1	Data Inspection & Audit	Profil data, missing value, duplikat, statistik numerik & kategorikal
2	Perbaikan tipe TotalCharges	String kosong menjadi NaN, tipe menjadi numerik, aturan tenure = 0 menjadi 0
3	Separate Features & Target	Pisahkan fitur dari Churn; customerID dibuang (identifier)
4	Train-Test Split (80-20, stratified)	Split SEBELUM fit preprocessing agar test tidak ikut mempengaruhi statistik
5	Missing Value Imputation	Median untuk numerical, modus untuk categorical (fit di train)
6	Categorical Encoding	Ordinal untuk Contract; OneHot untuk biner & nominal
7	Feature Scaling	StandardScaler untuk tenure, MonthlyCharges, TotalCharges
8	Imbalance Handling	class_weight / SMOTE, apply ONLY pada training data, bukan test data
9	Model Training & Evaluation	LogReg, KNN, Decision Tree dievaluasi pada test set asli

KEY PRINCIPLE - AVOID DATA LEAKAGE: train-test split HARUS dilakukan sebelum fitting preprocessing. Statistik (median, mean, std, kategori, tetangga SMOTE) hanya dihitung dari training data, dan test data diproses memakai statistik training. Ini mensimulasikan skenario nyata di mana test data adalah data "masa depan" yang belum pernah dilihat.

C. FOUR PREPROCESSING EXPERIMENTS & RESULTS

Kami melakukan eksperimen A-D untuk mengevaluasi dampak setiap langkah preprocessing. Model hanya alat observasi (bukan untuk mengejar skor terbaik); semua metrik dihitung pada test set asli, churn = kelas positif.

Exp	Algoritma	Preprocessing	Accuracy	Precision	Recall	F1-Score
A	KNN	Tanpa scaling (baseline)	76,22%	58,05%	36,73%	44,99%
B	KNN	+ StandardScaler	80,20%	66,55%	50,67%	57,53%
C	LogReg	Tanpa imbalance handling	81,48%	71,05%	50,67%	59,15%
D1	LogReg	+ class weight	75,09%	51,94%	78,82%	62,62%

D2	LogReg	+ SMOTE	76,01%	53,22%	77,48%	63,10%
----	--------	---------	--------	--------	--------	--------

1) Experiment A vs B: Impact of Scaling

Temuan: StandardScaler membuat KNN ROC-AUC naik dari 0,772 menjadi 0,830 dan accuracy naik dari 76,22% menjadi 80,20%. Pada Logistic Regression ROC-AUC hampir sama (0,860 vs 0,860), dan prediksi Decision Tree identik pada 100,0% data test.

Insight: KNN adalah distance-based algorithm sehingga fitur ber-range besar (TotalCharges) mendominasi jarak jika tidak di-scale. Decision Tree tidak terpengaruh karena split hanya bergantung pada urutan nilai fitur, bukan skala.

2) Experiment C vs D: Impact of Imbalance Handling

Temuan: Logistic Regression tanpa penanganan (C) memiliki accuracy 81,48% tetapi recall churn hanya 50,67%. Dengan class_weight (D1) recall naik menjadi 78,82%, dengan SMOTE (D2) menjadi 77,48%; precision turun ke 51,94% dan accuracy turun ke 75,09% (D1).

Insight: accuracy bukan metrik yang baik untuk imbalanced dataset. Trade-off-nya wajar: lebih banyak pelanggan churn terdeteksi walau ada false positive, dan bagi bisnis melewatkan pelanggan yang akan churn lebih merugikan. Prioritaskan recall/F1.

D. TREATMENT JUSTIFICATION

1) Missing Value Treatment

Question: mengapa TotalCharges diisi 0, bukan median/mean atau deletion?

- Median TotalCharges seluruh pelanggan = 1.627, sedangkan rata-rata MonthlyCharges pada baris kosong hanya 64,4; mengisi median berarti pelanggan baru dianggap sudah membayar ribuan
- Deletion menghapus 11 pelanggan baru dan menghilangkan informasi; aturan tenure = 0 -> 0 mempertahankan sample size dan bermakna secara bisnis

2) Encoding Method

- Label encoding pada atribut nominal (mis. PaymentMethod) memberi urutan semu yang menyesatkan model linear/jarak; One-Hot menghindarinya
- Contract memiliki urutan komitmen yang nyata, sehingga OrdinalEncoder dengan urutan eksplisit lebih tepat dan hemat dimensi

3) Scaling Method: StandardScaler vs MinMaxScaler

- StandardScaler (mean = 0, std = 1) cocok untuk Logistic Regression & KNN dan tidak terikat pada nilai min/max sehingga lebih stabil terhadap outlier
- MinMaxScaler (range [0,1]) sangat sensitif terhadap outlier karena memakai min dan max; scaling tidak diperlukan untuk algoritma tree

4) Imbalance Handling: SMOTE vs class_weight

- class_weight tidak mengubah data (sederhana); SMOTE membuat synthetic samples minority class dari interpolasi k-nearest neighbors, lebih baik daripada random over-sampling atau under-sampling yang membuang informasi
- SMOTE hanya pada training (4.141 : 1.493 -> 4.141 : 4.141); test tidak berisi data sintetis agar evaluasi realistis
- Keterbatasan: SMOTE standar menginterpolasi kolom dummy 0/1 sehingga muncul nilai pecahan; SMOTENC lebih tepat untuk data campuran

5) Data Leakage

- Kemungkinan 1: fit scaler/imputer/encoder pada seluruh data sebelum split (statistik test ikut terpakai). Kemungkinan 2: SMOTE sebelum split (sampel sintetis diturunkan dari data yang kelak menjadi test)
- Demonstrasi pada KNN: SMOTE sebelum split memberi recall 89,86% dan F1 80,87%, sedangkan cara yang benar (SMOTE hanya di train) memberi recall 80,43%

dan F1 59,06% - skor bocor terlihat lebih tinggi tetapi tidak dapat dipercaya

E. KEY FINDINGS & RECOMMENDATIONS

1) Data Quality Summary & Final Dataset Condition

- Missing Values: TotalCharges 11 records -> diisi 0 (tenure = 0)
- Duplicate Records: 0 exact duplicate -> tidak ada yang dihapus
- Feature Scaling: tenure vs TotalCharges (skala 72 vs 7.938) -> StandardScaler
- Class Imbalance: 2,77:1 -> class_weight / SMOTE (train saja)
- Final Dataset: train 5.634 baris (8.282 setelah SMOTE), test 1.409 baris, 38 fitur, 0 missing value, 8/8 uji validasi leakage lulus

2) Best Practices Recommendations

- Selalu lakukan data quality audit terlebih dahulu - periksa tipe data, bukan hanya isna() (missing bisa tersembunyi sebagai spasi)
- Train-test split sebelum preprocessing; bungkus semua langkah dalam satu Pipeline/ColumnTransformer agar konsisten dan reproducible
- Gunakan domain knowledge dalam imputasi, bukan hanya pendekatan statistik; scaling untuk model berbasis jarak/gradien
- Tangani imbalance hanya pada training data dan prioritaskan recall/F1/ROC-AUC dibanding accuracy

3) Next Steps

- Hyperparameter tuning & cross-validation dengan preprocessing di dalam Pipeline
- Coba SMOTENC
- Feature engineering & monitoring kualitas data di produksi